

Smooth and Nonsmooth Markowitz Portfolio Optimization

Lucas Ahou

Guerand Dewell

I. INTRODUCTION

Portfolio optimization is the core of modern quantitative finance. It helps investor solve the trade-off problem between maximizing expected returns from investments while managing risks. Originally, this problem was described as a smooth mean-variance problem introduced by Markowitz [1]. This problem minimizes the portfolio variance while targeting a certain expected return. This first problem, however, does not take *transaction cost* into account, therefore leading to a second version of this problem integrating this in the objective. This new problem is more realistic, though it is now non-smooth. Because of this key difference between the two models, the suitable optimization methods will differ from one another.

The aim of this report is thus to compare different methods for both models by analyzing the computational cost of the methods and their convergence—both empirically and using theoretical results—

II. DATA

As said in the introduction, we will work on historical data sampled from the S&P500 index. The dataset is described in a .csv file where we have access to the name of the different stocks, the date and the open, close and low/high prices at that date. We are actually only interested in the date, name and close prices in this dataset. In fact we are actually interested in finding the average return vector μ and the corresponding covariance matrix of returns Σ . The first step was to extract the close prices as a matrix where each row corresponds to a different date and each column corresponds to a different stock's name. We then had to compute the returns at each date. At a time t , the return r_t is given by:

$$r_t = (p_t - p_{t-1})/p_{t-1}$$

The return μ is then simply given by the average of the returns with respect to time, for each assets. The covariance matrix Σ is also simply given by the covariance matrix of the return matrix. A strong property of (sample) covariance matrices is that they are always square, symmetric and positive semi-definite. This property is mandatory to be able to use strong theoretical results for convergence because it makes our objective function convex (quadratic function with PSD matrix).

In our implementation, we also included a feature to select only n stocks within all the available stocks. This will allow us to compare the computational costs of the diverse methods as the dimension of the variable increases.

III. SMOOTH MODEL

The first model we will study is the *Smooth Markowitz model*. In this section, we will first present the formulation of the model and the meaning of its constituting parts. We will then present the three methods we will implement to solve this problem and that we will analyze later.

A. Model

The *Smooth Markowitz model* is mean-variance problem that was introduced by Markowitz[1] in 1952. It is defined as follows:

$$\min_{w \in \Delta} f(w) = \frac{1}{2} w^\top \Sigma w - \lambda w^\top \mu \quad (1)$$

where $w \in \mathbb{R}^n$, $\Sigma \in \mathbb{R}^n$ and $\mu \in \mathbb{R}^n$.

The vector of variables w represents the *weights* of our portfolio, i.e. the proportion of each assets that constitutes our portfolio. Because each component w_i of w represents a percentage of our portfolio, they must sum up to one in a realistic scenario. This is why we define the feasible set as the simplex:

$$\Delta = \{w \in \mathbb{R}^n : w_i \geq 0, \mathbf{1}^\top w = 1\}$$

The matrix Σ is the covariance matrix and μ is the average vector of the returns available in the available dataset. Therefore, the model aims to do the following:

1. The first term $\frac{1}{2} w^\top \Sigma w$ represents the variance of the portfolio's return. Intuitively, we want to minimize it as this is a measure of risk. In fact, a highly varying portfolio return means we, a lot of time, will encounter negative results. Thus minimizing this term allows to have a more consistent portfolio.
2. The second term $-\lambda w^\top \mu$ is there to target a maximum average return, while still minimizing the variance with the presence of the first term. Without this second term, the solution for this problem will only try to create a consistent portfolio and will thus not make a lot of profit. The $\lambda > 0$ constant is a hyperparameter which controls the risk. The bigger it is, the more the model will try to find a portfolio selection that maximizes the average return compared to minimizing the variance. Therefore, this parameter represents the risk we are willing to take before solving this problem. The greater it is, the higher the risk is.

Now that we have seen an overview of this model and explained its meaning, we will discuss some properties of this model.

First of all, the most important aspect is that the model is convex. In fact, the objective function is a sum between a positive semi-definite quadratic form (first term) and a linear function in the variables (second term). A sum of two convex functions is convex, so the objective is convex. The

feasible set is also convex. This can be easily checked with the following computations:

Let $w, v \in \Delta = \{w \in \mathbb{R}^n : w_i \geq 0, \sum_{i=1}^n w_i = 1\}$, $\gamma \in [0, 1]$ and $u := \gamma w + (1 - \gamma)v$:

1. $u_i = \gamma w_i + (1 - \gamma)v_i \geq 0$ because $w_i, v_i, \gamma, (1 - \gamma) \geq 0$ ✓
2. $\sum_i u_i = \sum_i \gamma w_i + (1 - \gamma)v_i = \gamma \sum_i w_i + (1 - \gamma) \sum_i v_i = \gamma + (1 - \gamma) = 1$ ✓

Thus $u \in \Delta$ and Δ is convex ■

Proof 1. Δ is convex

The reason why it is important is that the local minima of a convex problem is also **global**. This will ensure that every method that we implement will converge towards a minimum with the same objective value. It will thus not be necessary to compare the performance of the portfolio in terms of its returns. Another important aspect of convex problems is that it allows us to use strong properties of convergence for the different methods.

Regarding those results and the implementation of the methods, we need to discuss some key properties of this model. More precisely, we need to derive the gradient and the hessian of the objective function, its smoothness constant as well as the projection operator on the simplex

a) *Gradient and hessian of f* : We will simply derive the objective function f with respect to w to obtain the gradient:

$$\nabla f(w) = \Sigma w - \lambda \mu$$

and a second time to get the hessian:

$$\nabla^2 f(w) = \Sigma$$

Something to notice is that the computational complexity of a gradient evaluation is $\mathcal{O}(n^2)$ because it is a matrix-vector multiplication. However, in the context of the *coordinate descent* method, it is not needed to compute the whole gradient as we will see later.

b) *Smoothness constant*: The smoothness constant of f is needed for the *Projected Gradient Descent* for example to use a step size that gives us better guarantees of convergence. Here is the derivation of this constant:

$\forall w, v \in \mathbb{R}^n$, we have:

$$\begin{aligned} \|\nabla f(w) - \nabla f(v)\|_2 &= \|\Sigma w - \Sigma v\|_2 \leq \|\Sigma\| \cdot \|w - v\|_2 \\ &= |\lambda_{\max}(\Sigma)| \cdot \|w - v\|_2 \end{aligned}$$

$$\Rightarrow L = |\lambda_{\max}(\Sigma)| = \lambda_{\max}(\Sigma)$$

where we used the fact that Σ is PSD to simplify the last expression.

Derivation 1. Smoothness constant

c) *Projection on the simplex*: The projection on the simplex is used by every method presented in this report, except for the interior point method. Below, we derive the calculation of the projection on this set:

We want to solve:

$$P_{\Delta}(v) = \arg \min_{w \in \Delta} \|w - v\|_2^2$$

which can be formulated as a constrained quadratic minimization problem:

$$\min_{w \in \mathbb{R}^n} \frac{1}{2} \|w - v\|_2^2 \quad \text{s.t.} \quad \sum_{i=1}^n w_i = 1, w_i \geq 0$$

We now define the Lagrangian of this problem as follows:

$$\begin{aligned} \mathcal{L}(w, \theta, \alpha) &= \frac{1}{2} \sum_i (w_i - v_i)^2 - \theta \left(\sum_i w_i - 1 \right) \\ &\quad - \sum_i \alpha_i w_i \end{aligned}$$

where $\theta \in \mathbb{R}$ and $\alpha \in \mathbb{R}^n$ are the Lagrange multiplier associated with the equality and nonnegativity constraints respectively.

We now impose the KKT conditions [2]:

1. Stationarity:

$$\frac{\partial \mathcal{L}}{\partial w_i} = w_i - v_i - \theta - \alpha_i = 0 \Rightarrow w_i = v_i + \theta + \alpha_i$$

2. Primal feasibility:

$$w_i \geq 0, \quad \sum_{i=1}^n w_i = 1$$

3. Dual feasibility:

$$\alpha_i \geq 0$$

4. Complementary slackness:

$$w_i \alpha_i = 0$$

and we now have to solve them. First, we notice that from the complementary slackness:

$$\text{If } w_i > 0, \text{ then } \alpha_i = 0 \Rightarrow w_i = v_i + \theta$$

$$\text{If } w_i = 0, \text{ then } \alpha_i \geq 0 \Rightarrow v_i + \theta \leq 0$$

The second case shows that we do not violate any KKT conditions. We thus have:

$$w_i = \max(v_i + \theta, 0)$$

To determine θ , we use the equality constraint:

$$\sum_i \max(v_i + \theta, 0) = 1$$

This sum can also be expressed in the following manner:

$$\sum_{i: v_i + \theta > 0} v_i + \theta = 1$$

If we now define k as the number of indices that satisfy $v_i + \theta > 0$ and $v_{(i)}$ be the sorted components of v , we get:

$$\sum_{i: v_i + \theta > 0} v_i + \theta = \sum_{i=n-k+1}^n v_{(i)} + \theta = \sum_{i=n-k+1}^n v_{(i)} + k\theta$$

Derivation 2. Projection operator on the simplex

This gives us:

$$\theta = \frac{1 - \sum_{i=n-k+1}^n v(i)}{k}$$

This k also needs to satisfy:

$$v_{(n-k)} + \theta \leq 0 \quad \text{and} \quad v_{(n-k+1)} + \theta > 0$$

There is a unique k that satisfies this relation which gives us the correct theta.

Derivation 2. (cont.)

The number k -and thus the projection-can be implemented more efficiently using this algorithmic approach:

Input: Vector v to project

Output: $P_{\Delta}(v)$

Time complexity: $\mathcal{O}(n \log n)$

1. Sort v in descending order $\Rightarrow u_1 \geq u_2 \geq \dots \geq u_n$
2. $k \leftarrow \max \left\{ j \in \{1, \dots, n\} : u_j + \frac{1 - \sum_{i=1}^j u_i}{j} > 0 \right\}$
3. $\theta \leftarrow \frac{1}{k} \left(1 - \sum_{i=1}^k u_i \right)$
4. **return** w with $w_i = \max(v_i - \theta, 0)$

Algorithm 1. Projection computation

B. Projected Gradient Descent

We will now describe the first optimization method we will implemented for this model: the Projected Gradient Descent.

Given a point $w \in \Delta$, the idea is simply to perform a gradient descent step:

$$z = w - \alpha \nabla f(w)$$

where $\alpha > 0$ is our step size. However, it is clear that there is no guarantee that this new point z stays in the simplex after the step. We therefore will simply project this point on the simplex using Algorithm 1:

$$w^+ = P_{\Delta}(z) = P_{\Delta}(w - \alpha \nabla f(w))$$

The algorithm is described as follows:

Input: $w_0, \alpha > 0$

Output: approximate solution w_N

for $k = 0, 1, \dots, N - 1$ **do**

$$g_k \leftarrow \nabla f(w_k)$$

$$w_{k+1} \leftarrow P_{\Delta}(w_k - \alpha g_k)$$

end for

Algorithm 2. Projected Gradient Descent

Here-as well as for the other methods-multiple stopping criterion can be used. We can either, as described above, stop after a certain number of iterations, or we could stop after reaching a certain precision (either on the objective value as well as the iterates values). Although we do not have theoretical results that can improve the convergence of the

method, we have actually a lower-bound on the number of iterations to reach a certain precision if we take the right step-size. In fact, here we have a smooth objective function f with a smoothness constant $L = \lambda_{\max}(\Sigma)$. In this case, we can take the step size to be:

$$\alpha = \frac{1}{L}$$

Knowing that our function is also convex, we have the following convergence result:

$$f(w_k) - f(w_0) \leq \frac{L \|w_0 - w^*\|^2}{2k}, \quad \forall k \geq 1$$

Hence, our method has a rate of $\mathcal{O}(1/k)$, which also means that we need $\mathcal{O}(1/\varepsilon)$ to reach an ε -accuracy for the objective value.

C. Projected Gradient Descent with Momentum

Here we are going to slightly modify the previous algorithm by introducing the notion of momentum. In our previous algorithm, we were not taking advantage of the gradient of the previous iterates, i.e. $\nabla f(w_{k-1}), \nabla f(w_{k-2}), \dots, \nabla f(w_0)$. We could then introduce a momentum variable

$$m_{k+1} = \beta m_k + (1 - \beta) \nabla f(w_k), \quad \text{with } m_0 = 0$$

The projected momentum iterates becomes for an $w_k \in \Delta$

$$w_{k+1} = P_{\Delta}(w_k - \gamma m_{k+1})$$

with $\beta \in [0, 1]$

Momentum increases the influence of recent gradients while gradually vanishing older ones, which often accelerates convergence. The more detailed algorithm is described as follows :

Input: w_0, γ, β

Output: approximate solution w_N

$$m_0 = 0$$

for $k = 0, 1, \dots, N - 1$ **do**

$$g_k \leftarrow \nabla f(w_k)$$

$$m_{k+1} = \beta m_k + (1 - \beta) g_k$$

$$w_{k+1} = P_{\Delta}(w_k - \gamma m_{k+1})$$

end for

Algorithm 3. Projected Gradient Descent with Momentum

However, with this algorithm (especially with a fixed β), the rate of convergence has the same order $\mathcal{O}(\frac{1}{k})$ as the projected gradient descent in the worst-case scenario.

A better version of this algorithm in the context of convex and smooth functions is the Nesterov's Accelerated Gradient Method, whose algorithm is described below:

Input: w_0, L
Init: $w_{-1} = w_0, \alpha = \frac{1}{L}, \lambda_0 = \beta_0 = 0$
Output: approximate solution w_N
for $k = 0, 1, \dots, N - 1$ **do**
 $y_k \leftarrow w_k + \beta_k(w_k - w_{k-1})$
 $w_{k+1} \leftarrow y_k - \alpha \nabla f(y_k)$
 $\lambda_{k+1} \leftarrow \frac{1 + \sqrt{1 + 4\lambda_k^2}}{2}, \quad \beta_{k+1} \leftarrow \frac{\lambda_k - 1}{\lambda_{k+1}}$
end for

Algorithm 4. Projected Gradient Descent with Momentum

This gives us this theoretical bound:

$$f(y_k) - f^* \leq \frac{2L \|w_0 - w^*\|^2}{(k+1)^2}, \quad \forall k \geq 1$$

So our convergence rate is $\mathcal{O}(1/k^2)$, which is faster than gradient descent's one. This also implies that the number of iterations is $\mathcal{O}(1/\sqrt{\varepsilon})$ to reach a precision of ε on the objective value.

D. Projected Randomized Coordinate Descent

The projected gradient Coordinate descent is a slight variant of the projected gradient descent method where we are going to choose randomly a direction of descent in order to reduce our computation complexity for large problems. Considering $w_k \in \Delta$ it is then defined by :

$$w_{k+1} = P_{\Delta} \left(w_k - \alpha [\nabla f(w_k)]_{i_k} \right) e_{i_k}$$

with $i_k \sim \mathcal{U}\{1, \dots, n\}$

Is it smart to make deterministic choices for the coordinates? Is the answer the same in theory and in practice? Discuss it.

IV. NON-SMOOTH MODEL

A. Model

Key differences with the smooth case. What fundamental changes are you expecting? Do you verify them?

B. Projected Subgradient Method

C. Proximal Gradient Descent

Did you think of other methods? Why could they help solve the problem? What structure of the problem made you think of this method?

V. NUMERICAL RESULTS

What comparisons could you make between the different methods? Is it always a fair choice?

Can some methods be greatly improved compared to the theory?

Are some of them disappointing? Do you have an explanation?

In general, does a model (smooth or non-smooth) bring better solutions? What do you mean by better? Are the methods faster? Do the solutions have a particular structure? Is it normal?

VI. CONCLUSION

Summary.

REFERENCES

- [1] H. Markowitz, "Portfolio Selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [2] "Karush–Kuhn–Tucker conditions." [Online]. Available: https://en.wikipedia.org/wiki/Karush%E2%80%93Kuhn%E2%80%93Tucker_conditions